

TECHSPECIALIST LIMITED

System Documentation

Executive and technical reference for the TechSpecialist Limited application: architecture, data storage, configuration, features, API surface, security posture, and operations.

Last verified against codebase: 2026-07-13

Contents

00. Documentation Index

01. Executive Overview

02. Architecture

03. Data Model and Storage

04. Configuration and Integrations

05. Feature Walkthroughs — Index

06. Careers Board and Application Form

07. AI CV Screening

08. AI Voice Interview (Stage 2)

09. HR Portal

10. AI Readiness Assessment (Lead Magnet)

11. Admin Leads Dashboard

12. API Reference

13. Security, Auth, and Known Gaps

14. Deployment and Operations

TechSpecialist Limited — System Documentation

This is the reference documentation for the TechSpecialist Limited web application: the public marketing site, the AI-powered recruitment platform, and the AI Readiness Assessment lead-generation tool. It exists so that anyone — a new engineer, a future vendor, or a non-technical stakeholder — can understand what the system does, how it's built, where data lives, and how it's configured, without having to reverse-engineer it from the code.

Audience: Written in two layers. Each document opens with a plain-language summary for executives/non-technical readers, then goes into full technical depth for engineers. Read the summaries for the big picture; read the full documents when you need to build, debug, or hand this system off.

Last verified against codebase: 2026-07-13. This documentation describes what the code actually does, not what it was intended to do — including gaps and inconsistencies found along the way. If the code changes, this documentation will drift; treat it as a snapshot, and re-verify anything security- or data-critical before relying on it.

Contents

Doc	Covers
00-executive-overview.md	What the system is, who uses it, and the major components — no jargon.
01-architecture.md	How the two applications (Next.js frontend, Python backend) fit together, request flow, environments.
02-data-model-and-storage.md	Every database table and storage location, what's stored where and why.
03-configuration-and-integrations.md	Every environment variable, every third-party service, what each is for.
04-features/	End-to-end walkthroughs of each major feature, one file per feature.
05-api-reference.md	Every API route in both the frontend and backend, grouped by area.
06-security-and-known-gaps.md	How auth actually works, and specific, verified gaps worth knowing about.
07-deployment-and-operations.md	How to run it locally, how it's deployed, and known operational debt.

The system in one paragraph

TechSpecialist Limited's site is actually **two applications working together**: a Next.js 15 frontend (marketing pages, careers board, an HR back-office, an AI-readiness quiz, and a leads admin panel) and a separate Python/FastAPI backend that owns the recruitment data and runs the AI screening and AI voice-interview logic. The frontend talks to the backend over HTTP/WebSocket, and falls back to writing directly to Supabase for a couple of flows if the backend is unreachable. See [01-architecture.md](#) for the full picture.

Executive Overview

What this system is

TechSpecialist Limited's web presence is not just a marketing website — it's a working business application with three distinct products living under one domain:

1. **The public website** — homepage, services, case studies, insights (blog), consultation booking, legal pages, and a privacy/data-deletion page for a separate mobile app ("ICE", built for NSIA).
2. **An AI-powered recruitment platform** — a careers board where candidates apply for jobs, an AI that automatically screens every CV against the job requirements, an AI that conducts a first-round voice interview with shortlisted candidates, and an internal HR portal where staff manage jobs, review candidates, schedule final interviews, and see recruitment analytics.
3. **An AI Readiness Assessment** — a public, self-serve quiz that scores a visitor's organization on "AI readiness," used as a marketing lead-generation tool, plus an internal dashboard to review the leads it captures.

Who uses it, and how

User	What they do	Where
Website visitor / prospect	Browses services and case studies, books a discovery call, takes the AI Readiness quiz	Public pages
Job candidate	Finds a job, applies with CV + cover letter, later completes an AI voice interview by email link	<code>/careers</code> , <code>/apply</code> , <code>/assessment/[token]</code>
HR staff	Posts jobs, reviews AI-screened candidates, approves/rejects, schedules final interviews, views analytics, manages other HR accounts	<code>/hr/*</code> (login required)
Marketing/sales staff	Reviews AI Readiness quiz leads, exports them, marks them as followed up	<code>/admin/assessments</code>

The recruitment pipeline, in plain terms

This is the most complex part of the system, and the one that does the most work:

1. HR posts a job
↓
2. Candidate applies (CV + cover letter, no account needed)
↓
3. AI reads the CV against the job requirements and produces a score + written strengths/concerns
↓
4. HR reviews the AI's assessment and approves or rejects — this is a human decision, not automated
↓
5. Approved candidates get an emailed link to a short AI-conducted voice interview
↓
6. The AI interviews the candidate by voice, topic by topic, and produces a scorecard
↓
7. HR reviews the scorecard and schedules a final (human) interview, or rejects

The AI never makes a final decision — it only produces scores and recommendations that a human reviews at two checkpoints (step 4 and step 7). Every action taken on a candidate (approved, rejected, interview scheduled, flagged for suspicious behavior during the AI interview) is written to an audit log.

Major components (non-technical summary)

- **The website you see** is built with Next.js, a modern web framework, and is what visitors' and candidates' browsers load directly.
- **A separate backend service** (built in Python) does the actual work of storing recruitment data, running the AI screening, and running the AI voice interviews. The website talks to this backend behind the scenes.
- **Two databases exist:** the Python backend's own database (the primary store for jobs, applications, interviews, HR accounts), and a Supabase (cloud Postgres) database that the website falls back to for job listings and applications if the backend is temporarily unreachable. See [02-data-model-and-storage.md](#) for why there are two, and what that implies.

- **AI providers:** Azure OpenAI (GPT-4o) powers CV screening and the voice interview; Anthropic's Claude (with Groq as a fallback) generates the AI Readiness Assessment reports. These are different AI systems for different features.
- **Email** is sent through Resend (recruitment notifications, reports) and EmailJS (marketing lead notifications on the public site).

What to know before relying on this system

A full, itemized list of verified gaps is in [06-security-and-known-gaps.md](#), but the two most important for decision-makers:

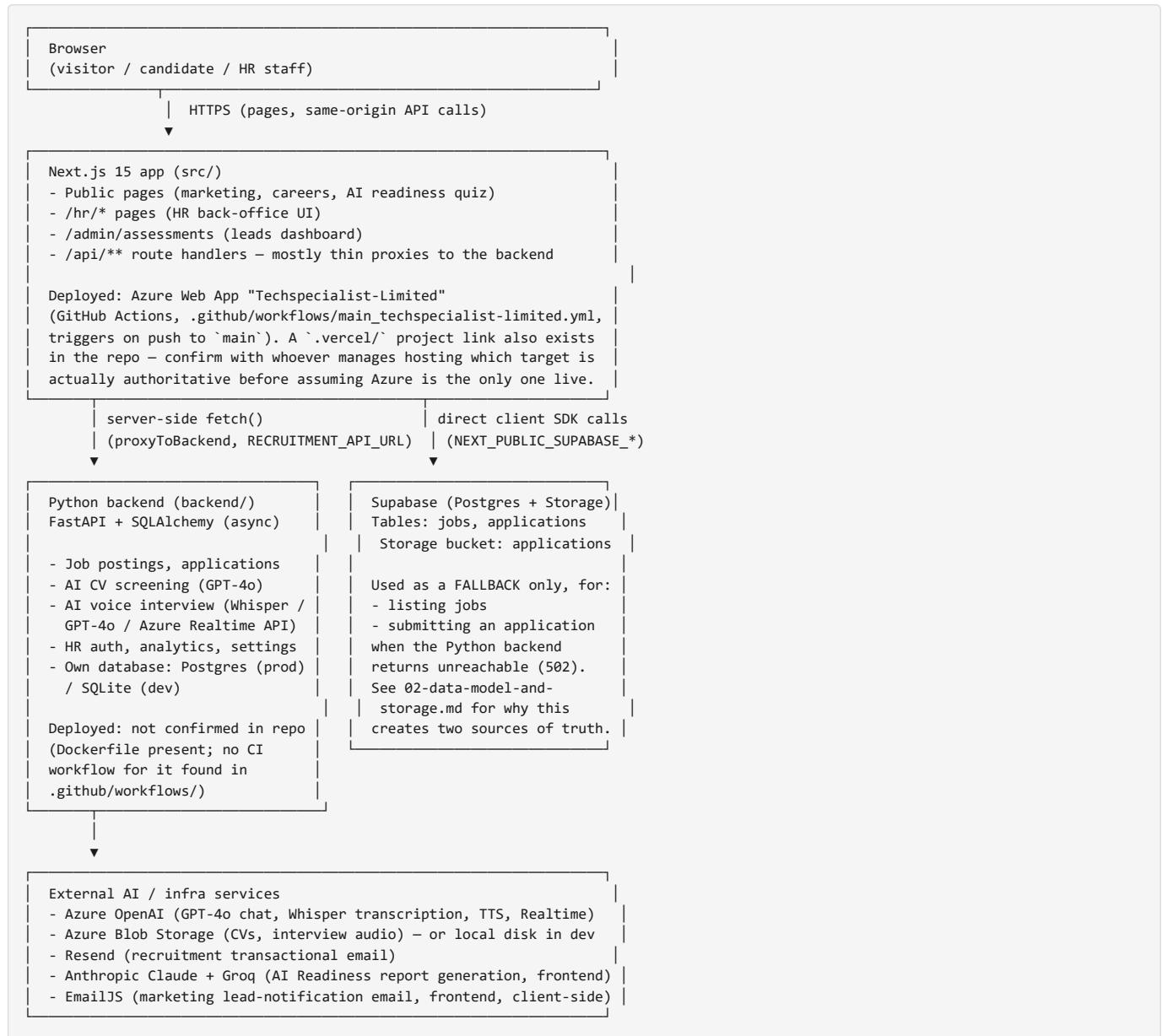
- **The AI Readiness Assessment leads dashboard (/admin/assessments) has no login or access control at all.** Anyone who finds the URL can view, export, or delete the captured leads.
- **Some recruitment data-entry paths bypass authentication** when the Python backend is unreachable, because the website's fallback path writes straight to Supabase without checking who's asking.

Neither of these affects candidates' CVs being screened incorrectly or interviews being scored unfairly — they are access-control gaps in administrative surfaces, not flaws in the AI logic itself.

Architecture

Summary

This is a **two-application system**, not one. A Next.js frontend serves everything a browser talks to; a separate Python (FastAPI) backend owns the recruitment database and runs the AI logic. The frontend calls the backend over HTTP and WebSocket. For two specific recruitment flows, the frontend also has a direct fallback to Supabase, so it can keep working (in a degraded way) if the Python backend is down.



Why two applications instead of one

The Next.js app could theoretically do everything (Next.js supports API routes). Instead, all the AI-heavy, stateful recruitment logic — CV screening, the voice-interview engine, HR auth, analytics — lives in a separate Python backend with its own database. The Next.js `api/recruitment/` **routes are, almost without exception**, thin proxies^{**}: they forward the request to the Python backend and return its response. The one shared helper for this is `src/app/api/recruitment/proxy.ts` (`proxyToBackend()`).

This split matters for anyone maintaining the system:

- **Recruitment business logic changes happen in `backend/`, not `src/`.** If you need to change how CV screening scores, how the interview state machine works, or how HR permissions are enforced, look in `backend/app/services/` and `backend/app/routers/`, not the Next.js API routes.
- **The Next.js layer's main jobs are:** rendering UI, proxying requests, and providing a Supabase-backed fallback for two specific write paths (job listing, application submission) so the public careers page degrades gracefully instead of hard-failing if the backend is briefly down.
- **The AI Readiness Assessment is entirely separate** from the recruitment platform's AI — it lives in the Next.js layer (`src/lib/ai-report.ts` , calling Anthropic/Groq directly) and in one backend router (`ai_readiness.py`) purely for storing lead submissions. It does not touch the recruitment database's core tables.

Request flow examples

A visitor loads the careers page: `GET /careers` → `src/app/careers/page.tsx` → `fetchJobs()` → `GET /api/recruitment/jobs` → `proxyToBackend()` → backend `GET /api/jobs`. If that fails, the Next.js route handler falls back to querying Supabase's `jobs` table directly.

A candidate submits an application: Browser → `POST /api/recruitment/applications` → proxied to backend `POST /api/applications` (protected by a shared `x-api-key` secret, not a user login) → backend extracts CV text, uploads files to storage, runs AI screening **synchronously in the same request** (see [07-deployment-and-operations.md](#) for why this is worth knowing), writes the `Application` + `AI Screening Result` rows, sends notification emails. If the backend call fails, the Next.js layer falls back to writing directly into Supabase's `applications` table and storage bucket instead — meaning that application would **never get AI-screened** (see [06-security-and-known-gaps.md](#)).

A candidate takes the AI voice interview: Browser → `wss://.../api/assessment/{token}/realtime-ws` (or the legacy HTTP/WebSocket engine) → backend proxies a live audio session to Azure OpenAI's Realtime API → on completion, backend scores the conversation and writes a `StageResult` row.

Environments

Environment	Frontend	Backend	Backend DB
Local dev	<code>npm run dev</code> (<code>next dev</code>)	<code>uvicorn app.main:app --reload</code> , <code>RECRUITMENT_API_URL=http://localhost:8000</code>	SQLite (<code>backend/recruitment.db</code>), <code>dev_mode true</code> , local-disk file storage under <code>backend/storage/</code>
Production	Azure Web App (Node/Next.js), per CI workflow	Not confirmed from this repo — see below	Postgres (per <code>backend/.env.example</code>), Azure Blob Storage

Gap worth flagging: there is no CI/CD workflow in `.github/workflows/` for the `backend/` service, and no `docker-compose.yml` tying frontend and backend together. The backend has a `Dockerfile` (`python:3.12-slim` , exposes port 8000) but how/where it's actually deployed in production is not determinable from this repository alone — confirm with whoever manages infrastructure and record the answer here.

A stray `out/` directory exists at the repo root (looks like a static export from `next export` / `output: 'export'` at some point), but the current `next.config.mjs` has no `output: 'export'` setting and `package.json`'s `build` script is plain `next build`. Treat `out/` as a stale build artifact unless someone confirms otherwise — it's not part of the live build pipeline.

Data Model and Storage

Summary

There are **two separate databases** and **two separate file-storage systems**, and they don't talk to each other:

- 1. **The Python backend's own database** — the primary, authoritative store for all recruitment data (jobs, applications, screening results, interviews, HR accounts). Postgres in production, SQLite locally.
- 2. **Supabase (Postgres)** — used by the Next.js frontend only as a fallback, holding a `jobs` table and an `applications` table that can drift out of sync with the backend's real tables.

This split exists because the frontend was built to degrade gracefully if the backend is briefly unreachable, but it means **an application submitted during a backend outage lands in Supabase, not the backend database, and never gets AI-screened or shows up in the HR portal**. This is the single most important storage fact to know about this system — see the callout at the bottom of this document.

1. Primary database (Python backend)

SQLAlchemy 2.0 async ORM. Dev default `sqlite+aiosqlite:///./recruitment.db` (file: `backend/recruitment.db`); production uses `postgresql+asyncpg:///...` via the `DATABASE_URL` env var. Tables are created from the ORM models on startup (`Base.metadata.create_all`); schema patches since initial deployment are applied by hand-rolled `ALTER TABLE` statements in `backend/app/migrate.py` (there is **no Alembic migration history** despite the `backend/alembic/` directory existing — it's empty; see [07-deployment-and-operations.md](#)).

All tables use a UUID primary key unless noted.

`job_postings` (model: `JobPosting` , file: `backend/app/models/job_posting.py`)

A job listing HR has created.

Field	Type	Notes
<code>id</code>	UUID PK	
<code>title</code>	string	
<code>description</code>	text	
<code>requirements</code>	text	
<code>department</code>	string(120)	
<code>location</code>	string(120)	
<code>type</code>	string(50)	default "Full-time"
<code>screening_instructions</code>	text	tells the AI what to weigh when scoring CVs for this role
<code>stage2_instructions</code>	text	system instructions for the AI voice interviewer
<code>stage2_questions</code>	text	JSON-encoded list of seed questions
<code>stage2_topic_labels</code>	text	JSON-encoded list of topic labels for the interview
<code>status</code>	string(20)	default "active"
<code>is_deleted</code>	bool	soft-delete flag
<code>is_closed</code>	bool	closed to new applications, but not deleted
<code>created_at</code> / <code>updated_at</code>	timestamp	

Relationship: one `JobPosting` → many `applications` .

`applications` (model: `Application` , file: `application.py`)

One candidate's application to one job. The central record of the recruitment pipeline.

Field	Type	Notes
id	UUID PK	
job_id	UUID FK → job_postings.id	
candidate_name , candidate_email	string(255)	
cv_url , cover_letter_url	text	storage URL (see storage section below)
cv_text , cover_letter_text	text, nullable	extracted plain text used for AI screening
status	string(20)	pending , approved , rejected , assessment_completed , assessment_flagged , interview_scheduled
stage	int	default 1 (1 = CV screen, 2 = AI voice interview, 3 = human interview)
assessment_token	string(255), unique, nullable	magic-link token emailed to the candidate for stage 2
assessment_sent_at , assessment_expires_at	timestamp, nullable	
created_at / updated_at	timestamp	

Relationships: belongs to one `JobPosting` ; has one `AI ScreeningResult` ; has many `StageResult` ; has one `ConversationSession` ; has many `Interview` .

`ai_screening_results` (model: `AI ScreeningResult` , file: `ai_result.py`)

The AI's stage-1 verdict on a CV, one per application.

Field	Type	Notes
id	UUID PK	
application_id	UUID FK → applications.id	
overall_score	float	
strengths , concerns , evidence	text, nullable	
raw_response	text, nullable	the AI's raw output, kept for auditability
created_at	timestamp	

`stage_results` (model: `StageResult` , file: `stage.py`)

The outcome of a completed stage-2 AI voice interview session.

Field	Type	Notes
id	UUID PK	
application_id	UUID FK	
stage_number	int	
status	string(20)	default "pending"
audio_url	text, nullable	
transcript	text, nullable	JSON-encoded conversation
score	float, nullable	
ai_feedback	text, nullable	JSON-encoded multi-dimension evaluation (communication, technical competency, confidence, problem-solving, relevance)
created_at / updated_at	timestamp	

`conversation_sessions` (model: `ConversationSession` , file: `conversation.py`)

The **live, in-progress** state of an AI voice interview — not the final result (that's `stage_results`).

Field	Type	Notes
id	UUID PK	
application_id	UUID FK	
status	text	default "in_progress"; also <code>completed</code> , <code>terminated_violation</code>
conversation_history	text	JSON list of {role, content, topic_label}
current_topic_index, turns_on_current_topic	int	interview state-machine bookkeeping
topics	text	JSON list of {label, seed_question}, copied from the job's <code>stage2_topic_labels</code> / <code>stage2_questions</code> at session start
engine	text	"legacy" (HTTP/WebSocket, Whisper+GPT-4o+TTS pipeline) or "realtime" (Azure OpenAI Realtime API, voice-to-voice)
created_at / updated_at	timestamp	

`interviews` (model: `Interview`, file: `interview.py`)

A **human-scheduled** final interview (stage 3) — distinct from the AI voice interview.

Field	Type	Notes
id	UUID PK	
application_id	UUID FK	
interview_type	string(20)	<code>physical</code> or <code>virtual</code>
scheduled_date, scheduled_time	string	
duration_minutes	int	default 60
location, meeting_link, interviewer_name, notes	nullable	
status	string(20)	default "scheduled"
created_at / updated_at	timestamp	

`audit_logs` (model: `AuditLog`, file: `audit_log.py`)

Compliance/history trail — every meaningful action taken on an application.

Field	Type	Notes
id	UUID PK	
application_id	UUID, indexed (not a formal FK)	
action	string(50)	e.g. <code>assessment_approved</code> , <code>rejected</code> , <code>interview_scheduled</code> , <code>interview_terminated_violation</code> , <code>application_cleared</code>
detail	text, nullable	
performed_by	string(255), nullable	
created_at	timestamp	

`hr_users` (model: `HrUser`, file: `hr_user.py`)

HR staff accounts — this backend has its own login system, entirely separate from Supabase.

Field	Type	Notes
id	UUID PK	

Field	Type	Notes
email	string(255), unique	
name	string(255)	
password_hash	string(255)	bcrypt
is_active	bool	default true
reset_token_hash , reset_token_expires_at	nullable	forgot-password / invite-a-new-user flow
created_at / updated_at	timestamp	

app_settings (model: `AppSetting` , file: `app_setting.py`)

Runtime-editable key/value store for HR-configurable branding, overlaid onto the app's in-memory settings at startup and whenever HR edits `/hr/settings` .

Field	Type	Notes
key	string(100) PK	company_name , brand_color , logo_url , sender_display_name , hr_notification_email
value	text	
updated_at	timestamp	

ai_readiness_assessments (model: `AiReadinessAssessment` , file: `ai_readiness_assessment.py`)

Lead-magnet quiz submissions. **Unrelated to the recruitment tables above** — no foreign keys connect them.

Field	Type	Notes
id	UUID PK	
email	string(255), indexed	
company_name	string(255)	default "Not provided"
scores	text	JSON dict, per-pillar scores
total_score	float	
max_score	float	default 75
level	string(50)	e.g. "AI Explorer", "AI Builder", "AI Accelerator", "AI Leader"
followed_up	bool	default false — toggled from <code>/admin/assessments</code>
created_at	timestamp	

2. Supabase (fallback database, used by the Next.js frontend)

Client: `src/lib/supabase.ts` , using the **anon key** (`NEXT_PUBLIC_SUPABASE_ANON_KEY`) — no service-role key is used anywhere in the frontend. Only two tables are touched from `src/` :

- **jobs** — read by `/careers` , `/careers/[id]` , `/apply` , and `GET /api/recruitment/jobs` , as a fallback when the backend's `GET /api/jobs` fails. Also written by the HR jobs API fallback (`src/app/api/recruitment/hr/jobs/route.ts`) when the backend is unreachable.
- **applications** — written by the legacy `POST /api/apply` route and by `POST /api/recruitment/applications` 's fallback path, when the backend is unreachable.

These are not guaranteed to have the same schema as the backend's `job_postings` / `applications` tables — they were built independently as a resilience fallback, not as a replica. Nothing in this repository keeps them in sync. Treat any data that ends up here (rather than in the backend's own database) as a case that needs manual reconciliation.

Supabase Storage

One bucket, **applications** , holding:

- `resumes/<uuid>.<ext>` — candidate CVs uploaded via the fallback path

- `cover-letters/<uuid>.<ext>` — candidate cover letters uploaded via the fallback path

3. Backend file storage (CVs, interview audio)

Independent of Supabase Storage. Controlled by `backend/app/services/storage.py`:

- **Local dev / no Azure Storage configured:** files are written to `backend/storage/<container>/` on local disk, referenced in the database as a `file:///<path>` URL. Confirmed present: `backend/storage/cvs/` (candidate CV/cover-letter text extracts, UUID-named `.txt` files).
- **Production (Azure Storage connection string configured):** files go to Azure Blob Storage, in containers named by `cvs_container_name` (default `cvs`) and `assessments_container_name` (for interview-answer audio).

The two-database problem — read this before changing anything

Because the Next.js frontend has an independent Supabase fallback for `jobs` and `applications`, **the backend's Postgres/SQLite database is not guaranteed to be the single source of truth** for those two entities. Concretely:

- A job created via `/hr/jobs/create` while the backend is reachable goes into the backend's `job_postings` table (correct, full-featured).
- A job listed on `/careers` when the backend happens to be down for that one request is read from Supabase's `jobs` table instead — which may be stale or missing rows the backend has.
- An application submitted while the backend is down goes into Supabase's `applications` table, **skips AI screening entirely**, and will not appear anywhere in the HR portal (which only reads from the backend database) until someone notices and manually moves it over.

If you're troubleshooting "a candidate says they applied but HR can't see them," check Supabase's `applications` table, not just the backend database. This is documented further in [06-security-and-known-gaps.md](#).

Configuration and Integrations

Summary

The system depends on roughly a dozen third-party services split across the frontend and backend: Supabase, Azure OpenAI (four different capabilities), Azure Blob Storage, Resend, Anthropic, Groq, and EmailJS. Every credential is environment-variable-driven except EmailJS's, which are hardcoded in source. This document lists every variable, what it's for, and where it's read — **values are intentionally not reproduced here**; find them in each environment's actual `.env` file or secrets manager.

Frontend environment variables (`.env.local` , or platform secrets in production)

Variable	Purpose	Read in
NEXT_PUBLIC_SUPABASE_URL	Supabase project URL	src/lib/supabase.ts
NEXT_PUBLIC_SUPABASE_ANON_KEY	Supabase anon (public) key — no service-role key is used	src/lib/supabase.ts
RECRUITMENT_API_URL	Base URL of the Python backend (default <code>http://localhost:8000</code>)	src/app/api/recruitment/proxy.ts and all recruitment API routes
RECRUITMENT_API_KEY / API_KEY	Shared secret sent as <code>x-api-key</code> when proxying the public application-submit call	src/app/api/recruitment/applications/route.ts
HR_EMAIL , HR_PASSWORD	Fallback HR login credentials used only if the backend is unreachable during login (see 06-security-and-known-gaps.md)	src/app/api/recruitment/hr/auth/route.ts
JWT_SECRET	Used only by the fallback token-minting path above — not real HMAC signing, string concatenation only	same file
GROQ_API_KEY	Fallback AI provider for AI Readiness report generation	src/lib/ai-report.ts
ANTHROPIC_API_KEY	Primary AI provider for AI Readiness report generation (Claude)	src/lib/ai-report.ts
RESEND_API_KEY	Sends the AI Readiness PDF report email	src/app/api/assessment/send-report/route.ts
NEXT_PUBLIC_ASSESSMENT_WS_URL	WebSocket URL for the AI voice-interview portals (default <code>ws://localhost:8000</code>)	RealtimeAssessmentPortal.tsx , LegacyAssessmentPortal.tsx

Present in `.env.local` but **not referenced anywhere in `src/`** — confirmed dead/unused:

- `ADMIN_SECRET_TOKEN` — appears to be a vestige of a planned-but-never-implemented `/admin` auth check. If you're asked "is `/admin/assessments` protected by this token," the answer is no — see [06-security-and-known-gaps.md](#).
- `VERCEL_OIDC_TOKEN` — Vercel-platform-injected, unrelated to app code.

Not environment-driven: `src/lib/emailjs.ts` hardcodes its public key, service ID, and template ID as string literals rather than reading them from env vars. This is lower-risk than it sounds (EmailJS's "public key" is designed to be exposed client-side), but it does mean rotating these values requires a code change, not a config change.

Backend environment variables (`backend/.env` , documented with placeholders in `backend/.env.example`)

Defined in `backend/app/config.py` (Pydantic settings, loaded from `.env`).

Group	Variables	Purpose
App	app_name , cors_origins , dev_mode	dev_mode toggles local-disk storage, console-logged emails, and silent TTS instead of hitting real Azure endpoints
Database	database_url	Postgres in prod, SQLite locally
Azure OpenAI — chat	azure_openai_endpoint , azure_openai_key , azure_openai_api_version , gpt4o_deployment_name	Powers CV screening and the "legacy" interview engine's conversation logic
Azure OpenAI — Whisper	azure_whisper_endpoint , azure_whisper_key , azure_whisper_api_version , whisper_deployment_name	Speech-to-text for the legacy voice interview engine

Group	Variables	Purpose
Azure OpenAI — TTS	<code>azure_tts_endpoint</code> , <code>azure_tts_key</code> , <code>azure_tts_api_version</code> , <code>tts_deployment_name</code> , <code>tts_voice</code>	Text-to-speech for the legacy voice interview engine
Azure OpenAI — Realtime	<code>azure_realtime_endpoint</code> , <code>azure_realtime_key</code> , <code>realtime_deployment_name</code> , <code>realtime_voice</code> , <code>realtime_vad_threshold</code> , <code>realtime_vad_prefix_padding_ms</code> , <code>realtime_vad_silence_duration_ms</code> , <code>realtime_max_session_seconds</code> , <code>realtime_interview_enabled</code>	Voice-to-voice interview engine (newer, feature-flagged by <code>realtime_interview_enabled</code>)
Interview tuning	<code>topic_time_limit_seconds</code> , <code>max_turns_per_topic</code>	Shared by both interview engines' state machines
Redis	<code>redis_url</code>	Declared but unused — see 07-deployment-and-operations.md ; no Celery task or worker process actually exists
Storage	<code>azure_storage_connection_string</code> , <code>cvs_container_name</code> , <code>assessments_container_name</code>	Azure Blob Storage; falls back to local disk if unset
Branding	<code>company_name</code> , <code>brand_color</code> , <code>logo_url</code>	Defaults, overridden at runtime by the <code>app_settings</code> DB table
Email	<code>resend_api_key</code> , <code>sender_email</code> , <code>sender_display_name</code> , <code>hr_notification_email</code>	All recruitment transactional email
Auth	<code>api_key</code> , <code>jwt_secret</code> , <code>hr_password</code>	<code>api_key</code> gates the public application-submit endpoint; <code>jwt_secret</code> signs real HR login JWTs (HS256); <code>hr_password</code> seeds the default HR account on first run
Frontend	<code>frontend_url</code>	Used to build links in outgoing emails (e.g. "review this candidate" links)

Third-party services — what each one is for

Service	Used by	Purpose
Supabase	Frontend	Fallback datastore for <code>jobs</code> / <code>applications</code> (see 02-data-model-and-storage.md); anon key only, no auth usage
Azure OpenAI (GPT-4o)	Backend	CV screening scoring, legacy interview conversation logic
Azure OpenAI (Whisper)	Backend	Speech-to-text for the legacy voice interview
Azure OpenAI (TTS)	Backend	Text-to-speech for the legacy voice interview
Azure OpenAI (Realtime API)	Backend	Voice-to-voice AI interview engine (the newer of the two interview engines)
Azure Blob Storage	Backend	Production file storage for CVs and interview audio
Resend	Both	Backend: all recruitment transactional email (approvals, rejections, invites, password resets). Frontend: AI Readiness Assessment PDF report delivery
Anthropic (Claude)	Frontend	Primary generator of AI Readiness Assessment report content (model: <code>claude-sonnet-4-6</code> , structured JSON output)
Groq	Frontend	Fallback generator for the same report if Anthropic isn't configured (model: <code>llama-3.3-70b-versatile</code>)
EmailJS	Frontend	Client-side-only marketing lead notifications (discovery call requests, AI Readiness quiz leads) — does not touch the backend at all

Two separate AI systems are in play and are easy to conflate: **Azure OpenAI** does all recruitment AI work (screening, interviews); **Anthropic/Groq** only generate the AI Readiness Assessment's marketing report. They share no code path.

Auth secrets — where they live and what they protect

See [06-security-and-known-gaps.md](#) for the full picture, but as a configuration reference:

- Backend HR JWTs are signed with `jwt_secret` (backend env) — this is the **real** auth mechanism.
- The frontend's `JWT_SECRET` / `HR_EMAIL` / `HR_PASSWORD` are a **separate, weaker fallback** that only activates if the frontend can't reach the backend during login. Keep these two sets of credentials in sync deliberately if you rotate one — they are not the same secret and rotating only one will

not fully lock out the fallback path.

Feature Walkthroughs

Each file traces one feature end-to-end: which pages/components are involved, which API calls fire, and what happens on the backend. Written so someone can find "where do I change X" without grepping the whole repo.

File	Feature
01-careers-and-application.md	Public job board and the candidate application form
02-ai-cv-screening.md	Automatic AI scoring of a submitted CV
03-ai-voice-interview.md	The AI-conducted stage-2 interview (two engines)
04-hr-portal.md	The internal HR back-office
05-ai-readiness-assessment.md	The public lead-generation quiz and its PDF report
06-admin-leads-dashboard.md	The AI Readiness leads admin panel

Careers Board and Application Form

What it does

Lets any visitor browse open roles and apply with a CV and optional cover letter — no account required.

Walkthrough

1. `/careers` (`src/app/careers/page.tsx`) loads the job list via `fetchJobs()` (`src/lib/recruitment-api.ts`), which calls `GET /api/recruitment/jobs` . That route proxies to the backend's `GET /api/jobs` ; if the backend is unreachable, it falls back to reading Supabase's `jobs` table directly. Only jobs with `status === 'active' && !is_closed` are shown.
2. `/careers/[id]` (`CareerDetailClient.tsx`) shows one job's full description and requirements, with a link to `/apply?id=<jobId>` .
3. `/apply` (`src/app/apply/page.tsx` , component `ApplyForm`) fetches the specific job (`fetchJob(jobId)`), then renders a 3-step form: candidate details → document upload (`src/components/recruitment/FileUploadZone.tsx`) → review.
4. On submit, `submitApplication(jobId, formData)` tries, in order:
 - `POST /api/recruitment/applications` → proxied to the backend's `POST /api/applications` (protected by the `x-api-key` shared secret, not a login) → backend extracts CV text, uploads files to storage, **runs AI screening synchronously**, writes `Application` + `AIScreeningResult` rows, sends confirmation/notification emails.
 - If that whole path throws, it falls back to the legacy `POST /api/apply` route — a pure Supabase insert with **no AI screening at all**.

Where to make changes

Change	File
Job board filtering/sorting	<code>src/app/careers/page.tsx</code>
Application form fields/steps	<code>src/app/apply/page.tsx</code>
What happens when an application is submitted (validation, storage, screening trigger)	<code>backend/app/routers/applications.py</code>
CV text extraction (PDF/DOCX parsing)	<code>backend/app/routers/applications.py</code> (uses <code>PyMuPDF</code> / <code>python-docx</code>)
Confirmation/notification email content	<code>backend/app/services/email_service.py</code> (<code>send_application_received_email</code> , <code>send_new_application_notification</code>)

Things worth knowing

- The `x-api-key` protecting `POST /api/applications` is a single shared secret, not per-user — it exists to keep the endpoint from being hit by arbitrary bots, not to authenticate the applicant.
- If the backend is down at submission time, the candidate's application is silently accepted into Supabase instead, and will **not** be AI-screened or visible in the HR portal. See [02-data-model-and-storage.md](#) and [06-security-and-known-gaps.md](#).

AI CV Screening

What it does

The moment a candidate submits an application, the backend has GPT-4o read the CV (and cover letter, if provided) against the job's `screening_instructions` and produce a score plus written feedback — before HR ever looks at it.

Walkthrough

1. Triggered inline, synchronously, inside `POST /api/applications` (`backend/app/routers/applications.py`) — not a background job (see [07-deployment-and-operations.md](#) for why that matters operationally).
2. `backend/app/services/ai_screening.py` calls Azure OpenAI GPT-4o with a structured-JSON-response prompt built from the CV text, cover letter text, and the job's requirements/screening instructions.
3. The response is normalized into: `overall_score` , `strengths` , `concerns` , `evidence` , `recommendation` .
4. `backend/app/workers/tasks.py` 's `run_screening()` upserts an `AIScreeningResult` row linked to the application.
5. If the AI call errors for any reason, the system does **not** fail the application — it falls back to a neutral score of 50 with a "needs manual review" note, so a candidate is never silently lost because of an AI outage.

Where to make changes

Change	File
Scoring prompt / what the AI weighs	<code>backend/app/services/ai_screening.py</code>
Per-job screening emphasis	<code>job_postings.screening_instructions</code> , editable from <code>/hr/jobs/create</code> and <code>/hr/jobs/[jobId]</code> (edit)
Fallback behavior on AI failure	<code>backend/app/services/ai_screening.py</code>
Where the score/feedback surfaces to HR	<code>src/app/hr/applicants/[applicationId]/page.tsx</code> , <code>src/app/hr/jobs/[jobId]/applicants/page.tsx</code>

Things worth knowing

- This step runs **in the request path** of the candidate's submission — the candidate's browser waits for GPT-4o to respond before getting a success confirmation. A slow or degraded Azure OpenAI response directly slows down the application form for the candidate.
- The `raw_response` field is kept on `AIScreeningResult` specifically so a human can audit exactly what the AI said, even if the parsed `strengths` / `concerns` fields don't fully capture it.

AI Voice Interview (Stage 2)

What it does

Once HR approves a candidate past the CV screen, the candidate gets an emailed magic link to a short AI-conducted voice interview: the AI asks questions on a set of topics, listens to spoken answers, and produces a scorecard.

Two engines exist — know which one is active

	Legacy engine	Realtime engine
Mechanism	Candidate speaks → Whisper transcribes → GPT-4o decides the next turn → Azure TTS speaks the reply. Turn-based.	Azure OpenAI Realtime API — true voice-to-voice, bidirectional streaming over WebSocket.
Backend router	<code>backend/app/routers/assessment.py</code> (HTTP) and <code>assessment_ws.py</code> (WebSocket variant, per-sentence streamed TTS)	<code>backend/app/routers/assessment_realtime_ws.py</code>
Frontend component	<code>LegacyAssessmentPortal.tsx</code>	<code>RealtimeAssessmentPortal.tsx</code>
Feature flag	Always available	Gated by backend config <code>realtime_interview_enabled</code>
<code>conversation_sessions.engine</code> value	"legacy"	"realtime"

`src/app/assessment/[token]/page.tsx` (`AssessmentPortalSwitcher`) fetches session metadata from `GET /api/recruitment/assessment/[token]` and picks the engine based on what the backend reports.

Walkthrough (both engines, conceptually)

- Candidate opens the emailed link: `/assessment/{token}`. The token is `Application.assessment_token`, a `secrets.token_urlsafe(32)` string — the sole credential for this flow, no login. It can expire (`assessment_expires_at`); an expired link returns HTTP 410 and the candidate sees an "expired" state.
- A `ConversationSession` row is created (or resumed), seeded with `topics` copied from the job's `stage2_topic_labels` / `stage2_questions`.
- The interview proceeds topic-by-topic. State machine rules (verified by `backend/tests/test_realtime_advance_topic.py`, the one real unit test in the backend):
 - A "follow-up" response stays on the current topic.
 - "Next topic" advances and resets the turn counter.
 - Hitting `max_turns_per_topic` forces a topic advance even if the model wanted a follow-up.
 - Advancing past the last topic marks the interview done.
 - An explicit "end interview" signal always ends it, regardless of topic position.
- Violation detection** (realtime engine only): if the candidate appears to leave the browser tab mid-interview, the session is flagged, the application status becomes `assessment_flagged`, and an audit log entry is written.
- On completion, `backend/app/services/conversation.py`'s `evaluate_full_conversation()` (legacy) or the realtime engine's equivalent produces a multi-dimension score (communication, technical competency, confidence, problem-solving, relevance, per-topic breakdown, recommendation), written to a new `StageResult` row.
- The interview auto-finalizes if it hits `realtime_max_session_seconds` (realtime engine) without an explicit end.

Where to make changes

Change	File
Interview state machine (topic advance rules)	<code>backend/app/services/realtime_interviewer.py</code> (<code>advance_topic()</code>) — has a real unit test, change it with matching test updates
Legacy engine conversation logic / prompts	<code>backend/app/services/conversation.py</code>
Realtime engine session config, system instructions, prompt-injection defenses	<code>backend/app/services/realtime_interviewer.py</code>

Change	File
Speech-to-text	<code>backend/app/services/audio_evaluation.py</code> (<code>transcribe_audio</code>)
Text-to-speech	<code>backend/app/services/tts.py</code>
Per-job interview topics/questions	<code>job_postings.stage2_topic_labels</code> / <code>stage2_questions</code> , editable from <code>/hr/jobs/create</code>
Interview timing limits	backend config: <code>topic_time_limit_seconds</code> , <code>max_turns_per_topic</code> , <code>realtime_max_session_seconds</code>

Things worth knowing

- The realtime engine's system instructions explicitly include prompt-injection/jailbreak defenses (`realtime_interviewer.py`) — worth reviewing before changing that prompt, since it's the one place actively defending against a candidate trying to manipulate the interviewer.
- `audio_evaluation.py` 's standalone `evaluate_transcript()` function appears to be legacy/unused — current routers call `conversation.py` 's full-conversation evaluator instead. Confirm before relying on it.
- Interview audio is uploaded to the `assessments` storage container (Azure Blob in prod, local disk in dev) — uploads appear to be best-effort/non-fatal if they fail, so don't assume every session has a recoverable audio file.

HR Portal

What it does

The internal back-office where HR staff post jobs, review AI-screened candidates, approve/reject, schedule final interviews, view analytics, and manage their own team's accounts.

Access

Gated by `src/app/hr/layout.tsx`, which redirects to `/hr/login` unless `isHRAuthenticated()` (a client-side check of `localStorage['hr_token']`). See [06-security-and-known-gaps.md](#) for the important caveat that this is a **UI-level** guard — the real enforcement is the backend's JWT check on each API call.

Pages

Route	Purpose
<code>/hr</code>	Dashboard — stat cards, active jobs
<code>/hr/login</code> , <code>/hr/reset-password</code>	Auth (the two pages exempted from the layout guard)
<code>/hr/jobs</code> , <code>/hr/jobs/create</code> , <code>/hr/jobs/history</code>	Create/manage/soft-delete/restore job postings
<code>/hr/jobs/[jobId]/applicants</code>	Applicants for one job
<code>/hr/applicants</code>	All applicants, searchable
<code>/hr/applicants/[applicationId]</code>	Full applicant detail: CV screening result, interview transcript/scores, approve/reject, schedule interview, resend assessment link, audit log
<code>/hr/interviews</code>	Interview calendar/list
<code>/hr/analytics</code>	Recruitment analytics — overview, pipeline funnel, interview stats, time-to-hire, trends
<code>/hr/documents</code>	View/download applicant CVs and cover letters
<code>/hr/settings</code>	Branding settings + HR team member management (invite, deactivate, delete)

Walkthrough — reviewing and advancing a candidate

- HR opens `/hr/applicants/[applicationId]`, which calls `fetchApplicantDetail()` → `GET /api/recruitment/hr/applicants/[applicationId]` → backend `GET /api/hr/applications/detail/{application_id}`.
- The page renders the AI screening score/strengths/concerns, and, if a stage-2 interview has happened, the transcript and scorecard (`TranscriptViewer`, `ScoreCircle`, `ScoreBar` in `src/components/recruitment/`).
- HR clicks approve or reject → `reviewApplication()` / `approveApplication()` → `POST /api/recruitment/hr/review|approve/[applicationId]` → backend `POST /api/hr/review/{id}` or `/api/hr/approve/{id}`. Approving past stage 1 generates a new `assessment_token`, sets `assessment_sent_at` / `assessment_expires_at`, and emails the candidate the stage-2 interview link. Rejecting sends a rejection email. Both write an `AuditLog` entry.
- HR can resend the assessment link (`resendAssessment()` → `POST /api/recruitment/hr/resend/[applicationId]`) if the candidate's link expired.
- After a successful stage-2 interview, HR schedules a final human interview (`createInterview()` → `POST /api/recruitment/hr/interviews`), which emails an invitation and sets `applications.status = 'interview_scheduled'`.

Team management (`/hr/settings`)

- Inviting a new HR user creates an `HrUser` row with an unusable placeholder password and emails a set-password (reset-token) link.
- Deactivating or deleting the **last remaining active HR account is blocked** by the backend, to prevent HR from locking itself out entirely.

Where to make changes

Change	File
Dashboard stats	<code>backend/app/routers/hr_review.py</code> (<code>GET /api/hr/stats</code>)
Applicant list/detail data shape	<code>backend/app/routers/hr_review.py</code>
Approve/reject/interview email content	<code>backend/app/services/email_service.py</code>
Analytics calculations	<code>backend/app/routers/analytics.py</code>
Branding fields available in Settings	<code>backend/app/services/settings_service.py</code> (<code>OVERRIDABLE_KEYS</code>)
Team member invite/deactivate/delete rules	<code>backend/app/routers/hr_settings.py</code>

Things worth knowing

- Every mutating HR action (approve, reject, schedule interview, clear applications) writes to `audit_logs` — `/hr/applicants/[applicationId]` surfaces this per-applicant via `fetchAuditLogs()`.
- `POST /api/recruitment/hr/clear/[jobId]` bulk-deletes all applications (and cascading screening/stage/conversation/interview rows) for a job. This is destructive and irreversible — the UI confirms before calling it using the app's custom `ConfirmDialog` component, not the browser's native `confirm()`.

AI Readiness Assessment (Lead Magnet)

What it does

A public, self-serve quiz at `/ai-readiness-assessment` that scores a visitor's organization on "AI readiness" across several pillars, shows them results, and offers to email them a full PDF report — capturing their email as a marketing lead in the process. This feature is **entirely independent of the recruitment platform** — different AI provider, different data, no shared tables.

Walkthrough

- `src/app/ai-readiness-assessment/page.tsx` runs a client-side phase state machine: `landing` → `questions` → `results`, persisting progress to `localStorage['ts-ai-assessment']` so a visitor can resume if they navigate away.
- `AssessmentLanding.tsx` — pillar selection. `AssessmentQuestions.tsx` (with `AssessmentStepper.tsx` / `AssessmentProgress.tsx`) — question flow, driven by question data in `src/data/assessment`. `AssessmentResults.tsx` — computes and displays the score.
- On reaching results, `sendAssessmentLeadEmail()` (`src/lib/emailjs.ts`) fires a client-side EmailJS notification, and the results are posted to `POST /api/assessment` → proxied to backend `POST /api/ai-readiness/submit`, which stores an `AiReadinessAssessment` row.
- If the visitor requests the full report, `POST /api/assessment/send-report` runs:
 - `generateAIReport()` (`src/lib/ai-report.ts`) — calls Anthropic Claude (`claude-sonnet-4-6`, strict JSON-schema output) to write the report content; falls back to Groq (`llama-3.3-70b-versatile`) if Anthropic isn't configured; returns `null` if neither is.
 - `ReportDocument` (`src/lib/pdf-report.tsx`, built with `@react-pdf/renderer`) renders that content into a formatted PDF (executive summary, priority gaps, pillar analysis, action plan, success metrics, branded footer).
 - The PDF is emailed via Resend from `reports@techspecialistlimited.com`.
 - This work is deferred with Next.js's `after()` so the visitor's HTTP request returns immediately and the slow AI+PDF+email work happens after the response.

Where to make changes

Change	File
Quiz questions/pillars	<code>src/data/assessment</code>
Scoring logic	<code>AssessmentResults.tsx</code>
Report content/prompt	<code>src/lib/ai-report.ts</code> (<code>buildPrompt</code>)
PDF layout/branding	<code>src/lib/pdf-report.tsx</code>
Lead storage	<code>backend/app/routers/ai_readiness.py</code>

Things worth knowing

- This is the only feature in the codebase that calls Anthropic or Groq — everything else AI-related (screening, interviews) goes through Azure OpenAI.
- If neither `ANTHROPIC_API_KEY` nor `GROQ_API_KEY` is configured, report generation silently returns `null` — confirm the request-flow handles that gracefully (i.e. doesn't send a broken email) before assuming this path always works.

Admin Leads Dashboard

What it does

`/admin/assessments` lists everyone who has completed the AI Readiness Assessment quiz, so marketing/sales can follow up. Supports filtering by readiness level, CSV export, marking a lead as "followed up," and deleting a lead.

Walkthrough

1. On mount, `src/app/admin/assessments/page.tsx` calls `GET /api/admin/assessments` (`src/app/api/admin/assessments/route.ts`), which proxies to backend `GET /api/ai-readiness/results`.
2. The table shows each lead's email, company, score, level (`AI Explorer` / `AI Builder` / `AI Accelerator` / `AI Leader`), and follow-up status.
3. **Filter by level** — client-side.
4. **CSV export** — built client-side from the already-fetched data (a `Blob`), no separate export endpoint.
5. **Toggle followed-up** — `PATCH /api/admin/assessments` → backend `PATCH /api/ai-readiness/results/{id}`.
6. **Delete** — confirmed via the app's `ConfirmDialog` component, then `DELETE /api/admin/assessments` → backend `DELETE /api/ai-readiness/results/{id}`.

Where to make changes

Change	File
Table columns / filters	<code>src/app/admin/assessments/page.tsx</code>
CSV export format	same file
Lead list/update/delete logic	<code>backend/app/routers/ai_readiness.py</code>

This is the most important thing to know about this feature

There is no authentication or authorization anywhere in this path. No `layout.tsx` or guard component wraps `/admin`, `src/app/api/admin/assessments/route.ts` performs no auth check before proxying, and the backend's `/api/ai-readiness/*` router requires no auth either. Anyone who discovers the URL can view, export, or delete every captured lead.

A `ADMIN_SECRET_TOKEN` environment variable exists in `.env.local`, suggesting access control was planned, but it is not referenced anywhere in the codebase — it does nothing today.

If this dashboard holds real, current lead data, treat closing this gap as a priority — see [06-security-and-known-gaps.md](#) for the fix options.

API Reference

How to read this

The frontend (`src/app/api/`) and backend (`backend/app/routers/`) each expose their own routes. Almost every frontend recruitment route is a thin proxy to a backend route via `proxyToBackend()` (`src/app/api/recruitment/proxy.ts`), forwarding the `Authorization` header and any body/query params as-is. Where a frontend route also has a Supabase fallback, it's noted — those fallbacks are the ones without a backend-equivalent auth check (see [06-security-and-known-gaps.md](#)).

Auth column values: **none** (public), **api-key** (`x-api-key` shared secret), **HR JWT** (backend `verify_hr_token`), **token** (candidate magic-link token).

Frontend routes (`src/app/api/**`)

Route	Backend target	Fallback	Auth (as enforced)
POST <code>/api/apply</code>	— (legacy, Supabase-only)	n/a — this *is* the fallback	none
GET/PATCH/DELETE <code>/api/admin/assessments</code>	<code>/api/ai-readiness/results/{id}</code>	none	none
POST <code>/api/assessment</code>	<code>/api/ai-readiness/submit</code>	none	none
POST <code>/api/assessment/send-report</code>	— (calls Anthropic/Groq + Resend directly)	none	none
GET/POST <code>/api/recruitment/jobs</code> , GET <code>/api/recruitment/jobs/{jobId}</code>	<code>/api/jobs/{id}</code>	Supabase <code>jobs</code> (GET only)	none
POST <code>/api/recruitment/applications</code>	<code>/api/applications</code>	Supabase <code>applications</code> insert + storage upload	api-key (backend only; fallback has none)
GET/POST <code>/api/recruitment/assessment/{token}</code>	<code>/api/assessment/{token}/{action}</code>	none	token
POST <code>/api/recruitment/hr/auth</code>	<code>/api/auth/login</code>	hardcoded <code>HR_EMAIL</code> / <code>HR_PASSWORD</code> check	none (this route *issues* the JWT)
POST <code>/api/recruitment/hr/auth/change-password</code>	<code>/api/auth/change-password</code>	none	HR JWT
POST <code>/api/recruitment/hr/auth/forgot-password</code>	<code>/api/auth/forgot-password</code>	none	none (by design — must work pre-login)
POST <code>/api/recruitment/hr/auth/reset-password</code>	<code>/api/auth/reset-password</code>	none	reset token
GET/POST <code>/api/recruitment/hr/jobs</code> , GET/PUT <code>/api/recruitment/hr/jobs/{jobId}</code>	<code>/api/jobs</code> (POST/PUT)	Supabase <code>jobs</code> read/write	HR JWT (backend only; fallback has none)
PUT <code>/api/recruitment/hr/jobs/{jobId}/soft-delete</code> , <code>/restore</code>	<code>/api/jobs/{id}/soft-delete</code> , <code>/restore</code>	none	HR JWT
GET <code>/api/recruitment/hr/jobs/history</code>	<code>/api/jobs/history</code>	none	HR JWT
GET <code>/api/recruitment/hr/applications/{jobId}</code>	<code>/api/hr/applications/{job_id}</code>	none	HR JWT
GET <code>/api/recruitment/hr/applicants/{applicationId}</code>	<code>/api/hr/applications/detail/{id}</code>	none	HR JWT
GET <code>/api/recruitment/hr/applicants/{applicationId}/document</code>	<code>/api/hr/applications/{id}/document</code>	none	HR JWT
POST <code>/api/recruitment/hr/review/{applicationId}</code>	<code>/api/hr/review/{id}</code>	none	HR JWT

Route	Backend target	Fallback	Auth (as enforced)
POST /api/recruitment/hr/approve/[applicationId]	/api/hr/approve/{id}	none	HR JWT
POST /api/recruitment/hr/resend/[applicationId]	/api/hr/resend/{id}	none	HR JWT
POST /api/recruitment/hr/clear/[jobId]	/api/hr/clear/{job_id}	none	HR JWT
GET /api/recruitment/hr/audit-logs/[applicationId]	/api/hr/audit-logs/{id}	none	HR JWT
GET /api/recruitment/hr/stats	/api/hr/stats	none	HR JWT
GET /api/recruitment/hr/analytics/{overview,interviews,pipeline,time-to-hire,trends}	/api/hr/analytics/*	none	HR JWT
GET/POST /api/recruitment/hr/interviews , GET/PUT /api/recruitment/hr/interviews/[interviewId]	/api/hr/interviews[/]{id}]	none	HR JWT
GET /api/recruitment/hr/interviews/{all,upcoming}	/api/hr/interviews/{all,upcoming}	none	HR JWT
GET /api/recruitment/hr/interviews/by-application/[applicationId]	/api/hr/interviews/by-application/{id}	none	HR JWT
GET/PUT/POST /api/recruitment/hr/settings	/api/hr/settings	none	HR JWT
GET/POST /api/recruitment/hr/users , PATCH/DELETE /api/recruitment/hr/users/[userId]	/api/hr/users[/]{id}]	none	HR JWT

Backend routes (backend/app/routers/**)

jobs.py — prefix /api/jobs

- POST /api/jobs — create a job posting (HR JWT)
- GET /api/jobs — list jobs (public; show_all / deleted query flags)
- GET /api/jobs/history — soft-deleted jobs (public)
- GET /api/jobs/{job_id} — one job + applicant count (public)
- PUT /api/jobs/{job_id} — update status/departement/location/type/is_closed (HR JWT)
- PUT /api/jobs/{job_id}/soft-delete / /restore (HR JWT)

applications.py — prefix /api/applications

- POST /api/applications — candidate submits (multipart: job_id, name, email, cv, optional cover letter; requires x-api-key)
- GET /api/applications/{application_id} — fetch one (public)

hr_review.py — prefix /api/hr

- GET /api/hr/stats — dashboard counts (HR JWT)
- GET /api/hr/applications/{job_id} — applicants for a job, sorted by score (HR JWT)
- GET /api/hr/applications/detail/{application_id} — full applicant detail (HR JWT)
- POST /api/hr/approve/{application_id} — advance stage, send assessment invite (HR JWT)
- POST /api/hr/review/{application_id} — approve or reject (HR JWT)
- POST /api/hr/resend/{application_id} — regenerate/resend assessment link (HR JWT)
- POST /api/hr/clear/{job_id} — bulk-delete a job's applications and related rows (HR JWT)
- GET /api/hr/applications/{application_id}/document — stream CV/cover letter (HR JWT)
- GET /api/hr/audit-logs/{application_id} — audit trail (HR JWT)

analytics.py — prefix /api/hr/analytics (all HR JWT)

- GET /overview , /interviews , /pipeline , /time-to-hire , /trends

interviews.py — prefix /api/hr/interviews (stage-3 human interviews, all HR JWT)

- `POST /` — schedule, sends invitation email, logs audit
- `GET /by-application/{application_id}, /upcoming, /all`
- `PUT /{interview_id}` — update, logs audit

assessment.py — prefix `/api/assessment` (legacy HTTP engine, token auth)

- `GET /{token}` — session metadata (410 if expired)
- `POST /{token}/start` — begin/resume, returns streamed TTS audio
- `POST /{token}/respond` — candidate audio in, transcribe → next AI turn → streamed TTS reply
- `POST /{token}/end` — force-end and score

assessment_ws.py — prefix `/api/assessment` (legacy engine, WebSocket)

- `WS /{token}/ws` — bidirectional streaming audio, per-sentence TTS

assessment_realtime_ws.py — prefix `/api/assessment` (Realtime engine)

- `WS /{token}/realtime-ws` — live voice-to-voice proxy to Azure OpenAI Realtime API, gated by `realtime_interview_enabled`

auth.py — prefix `/api/auth` (HR login)

- `POST /login` — returns JWT (HS256, 8h)
- `POST /forgot-password` — always returns a generic success message (avoids user enumeration)
- `POST /reset-password` — consumes reset token
- `POST /change-password` (HR JWT)

hr_settings.py — prefix `/api/hr`

- `GET/PUT /settings` — branding/notification config (HR JWT)
- `GET/POST /users` — list / invite HR accounts (HR JWT)
- `PATCH/DELETE /users/{user_id}` — activate/deactivate/delete, blocks removing the last active account (HR JWT)

ai_readiness.py — prefix `/api/ai-readiness` (no auth on any of these)

- `POST /submit` — public quiz submission
- `GET /results` — list all submissions
- `PATCH /results/{assessment_id}` — mark followed up
- `DELETE /results/{assessment_id}`

Misc

- `GET /health` — liveness check, returns `{"status": "ok"}`

Security, Auth, and Known Gaps

Summary for non-technical readers

There are two separate login systems in this application (one for HR staff, effectively none for the AI-readiness leads admin panel), and neither uses Supabase or Firebase's built-in authentication — both are custom-built. The HR system is reasonably solid when the Python backend is reachable, but has a **weaker fallback path** that activates during backend outages. The leads admin panel currently has **no access control at all**. This document lists what's verified true in the code today, so decisions about what to fix (and in what order) can be made with facts rather than assumptions.

How HR authentication actually works

1. **Real path (backend reachable):** `POST /api/recruitment/hr/auth` proxies to backend `POST /api/auth/login`, which checks the submitted password against a bcrypt hash in the `hr_users` table and, on success, issues a genuine JWT (HS256, signed with the backend's `jwt_secret`, 8-hour expiry). Every subsequent HR-scoped backend endpoint validates this JWT via `verify_hr_token()` (`backend/app/auth.py`).
2. **Fallback path (backend unreachable — HTTP 502):** `src/app/api/recruitment/hr/auth/route.ts` checks the submitted credentials against the frontend's own `HR_EMAIL` / `HR_PASSWORD` env vars (defaults `hr@company.com` / `admin123` if unset — **check these are actually set to non-default values in production**), and mints a token that **looks like a JWT but isn't**: `signature = base64url(JWT_SECRET + header + payload)`, i.e. string concatenation encoded, not HMAC-SHA256. This token would not pass a real JWT signature check, but nothing in the frontend re-validates it either — the frontend simply trusts it exists.
3. `src/lib/auth.ts` stores whichever token was issued in `localStorage['hr_token']` and attaches it as `Authorization: Bearer <token>` to every subsequent HR API call (`recruitment-api.ts`'s `authHeaders()`).
4. ****The `/hr/*` page guard (`src/app/hr/layout.tsx`) only checks that *some* token exists in `localStorage` **** — it does not verify it's valid. It is a UI-rendering guard, not a security boundary. The actual security boundary is the backend's `verify_hr_token()` on each API call.

The gap this creates

When a mutating HR route has a Supabase fallback (currently: `POST/PUT /api/recruitment/hr/jobs*`), that fallback code path performs **no auth check whatsoever** before writing to Supabase — it doesn't even look at the `Authorization` header. In practice, this means: if the backend is down, anyone who can reach `/api/recruitment/hr/jobs` (not just logged-in HR staff) can create or modify a job posting. This is a narrower gap than it sounds, because it only matters during a specific failure mode (backend down), but it's real and worth fixing — either by having those fallback routes also check for a valid token, or by removing the write fallback and simply failing the request when the backend is unreachable.

The AI Readiness leads admin panel has no auth at all

`/admin/assessments`, `src/app/api/admin/assessments/route.ts`, and backend `backend/app/routers/ai_readiness.py`'s `/api/ai-readiness/*` endpoints have **zero authentication or authorization** at any layer. Anyone who discovers the URL can:

- View every captured lead (email, company name, score)
- Export all of it as CSV
- Mark leads as followed up
- **Delete leads permanently**

An `ADMIN_SECRET_TOKEN` env var exists in `.env.local`, which strongly suggests this was meant to be protected and the wiring was never finished. This is the single highest-priority finding in this documentation set if the leads data is real and current.

Straightforward fix options (not yet implemented — for whoever picks this up):

- Reuse the existing HR JWT mechanism: require the same `Authorization: Bearer` header and `verify_hr_token()` check on the backend's `/api/ai-readiness/*` routes, and add the same `layout.tsx`-style client guard to `/admin`.
- Or, simplest: check `ADMIN_SECRET_TOKEN` as a shared secret (like the recruitment `x-api-key` pattern) if a full login isn't wanted for this one panel.

Other verified auth mechanisms (working as designed)

- **Public application submission** (`POST /api/applications`): a static shared secret via `x-api-key`, checked against backend `settings.api_key`. This isn't candidate authentication — it's a bot/abuse gate on a public form, and that's an appropriate use for a shared secret.
- **Candidate voice interview** (`/api/assessment/{token}/...`): a single-use-context magic-link token (`secrets.token_urlsafe(32)`), no password. Appropriate for a one-shot, emailed, time-limited flow — just be aware anyone who has the email link has the interview access, so link expiry

(`assessment_expires_at`) is the only real time boundary.

- **Password reset** (`POST /api/auth/forgot-password`): always returns a generic success message regardless of whether the email exists, correctly avoiding user enumeration.

Data-integrity gap: the Supabase fallback and AI screening

Covered in depth in [02-data-model-and-storage.md](#), but worth restating here because it's a data-integrity issue with a security-adjacent cause (an unauthenticated-by-design fallback path): an application submitted while the backend is down lands in Supabase, **skips AI screening**, and is invisible to the HR portal until manually reconciled. This isn't a confidentiality/access problem, but it is a silent-data-loss-shaped problem worth the same level of attention.

What is **not** a gap (confirmed by code, not assumption)

- No Supabase Auth or Firebase Auth is used anywhere for HR or admin — this is intentional custom auth, not a misconfigured integration.
- The Supabase client only ever uses the anon key; no service-role key is exposed anywhere in `src/`.
- Password reset tokens are hashed (SHA-256) before storage (`hr_auth_service.py`) with a 1-hour TTL — not stored in plaintext.
- HR passwords are bcrypt-hashed, not plaintext, in the `hr_users` table.

Recommended priority order

1. **Add auth to `/admin/assessments` and its API routes** — currently fully open, highest exposure for the least effort to fix.
2. **Close or auth-gate the Supabase fallback write paths for HR jobs** — narrower exposure (only during backend outages) but touches production job data.
3. **Confirm `HR_EMAIL` / `HR_PASSWORD` are not left at their code-level defaults** (`hr@company.com` / `admin123`) in whatever environment serves production traffic.
4. **Decide whether the fallback JWT-like token minting is acceptable long-term**, or whether the frontend should simply fail login when the backend is unreachable rather than issuing a weaker credential.

Deployment and Operations

Summary

The frontend has a clear CI/CD path to Azure Web App. The backend does not — it has a `Dockerfile` but no CI workflow in this repository, so how and where it actually runs in production isn't determinable from the code alone. There's also declared-but-unused infrastructure (Celery/Redis) worth knowing about before assuming background job processing exists.

Running locally

Frontend:

```
npm install
npm run dev0      # next dev – note: not the default "dev" script name
```

Requires `.env.local` with, at minimum, `RECRUITMENT_API_URL=http://localhost:8000` and the Supabase keys for the fallback paths to work.

Backend:

```
pip install -r requirements.txt
uvicorn app.main:app --reload
```

Requires `backend/.env` (see `backend/.env.example` for the full list). With no `DATABASE_URL` override, it defaults to a local SQLite file (`backend/recruitment.db`) and `dev_mode`, which also switches file storage to local disk and email sending to console-logging instead of calling Resend.

On every startup (`app/main.py` 's lifespan hook), the backend:

1. Creates any missing tables (`Base.metadata.create_all`).
2. Runs `run_migrations()` — hand-rolled `ALTER TABLE` statements for columns added after initial deployment (see below).
3. Runs `seed_defaults()` — creates a default HR account (`hr@company.com` , password from `HR_PASSWORD`) and default `app_settings` rows if they don't exist.

Deployment

Frontend: `.github/workflows/main_techspecialist-limited.yml` builds and deploys to an **Azure Web App** named `Techspecialist-Limited` on every push to `main` (via `azure/webapps-deploy`). Build-time secrets injected: `NEXT_PUBLIC_SUPABASE_URL` , `NEXT_PUBLIC_SUPABASE_ANON_KEY` , `NEXT_PUBLIC_ASSESSMENT_WS_URL` .

A `.vercel/repo.json` project link also exists in the repo, which implies a Vercel project may also be (or have been) connected. **Confirm with whoever manages hosting which one is the live, authoritative deployment** before making assumptions in an incident — don't assume Azure is the only place this is running.

Backend: has a `Dockerfile` (`python:3.12-slim` , installs `requirements.txt` , runs `uvicorn app.main:app --host 0.0.0.0 --port 8000`), but no matching GitHub Actions workflow exists in `.github/workflows/` . Where the backend is actually hosted in production is not recorded anywhere in this repository — **this is a gap in the documentation, not just the deployment setup: find out and record it here.**

Database migrations — read before touching the schema

Despite a `backend/alembic/` directory existing, **it is completely empty** — no `alembic.ini` , no `env.py` , no versioned migration files, and no Alembic import anywhere in the code. This project does **not** use Alembic in practice.

The actual mechanism is `backend/app/migrate.py` , run automatically on every app startup:

- New tables are created automatically from the SQLAlchemy models.
- New *columns* on existing tables are added via dialect-aware `ALTER TABLE` statements hardcoded in `run_migrations()` (a separate SQLite branch and Postgres branch). As of this writing, these patch in: `job_postings.is_deleted/is_closed/department/location/type` , `applications.is_archived/assessment_sent_at/assessment_expires_at` , `conversation_sessions.engine` .

Practical implication: if you need to add a new column to an existing table, you add it to the model *and* add a matching `ALTER TABLE ... ADD COLUMN` line to `run_migrations()` — there's no `alembic revision --autogenerate` workflow to lean on. If you need to drop a column, rename a

column, or do anything more complex than "add a nullable column," this hand-rolled approach won't help you and you'll need to write the migration by hand (and probably introduce real Alembic at that point).

Background jobs: declared but not implemented

`requirements.txt` lists `celery[redis]` and `redis`, and `config.py` defines `redis_url`. **No Celery app, task decorator, beat schedule, or worker process exists anywhere in the codebase.** The one function that looks like a background task — `backend/app/workers/tasks.py`'s `run_screening()` — is called directly and synchronously (`await run_screening(...)`) inline inside `POST /api/applications`, blocking that HTTP request until the GPT-4o call completes.

Practical implications:

- Don't assume there's a job queue you can push work onto — there isn't one yet, despite the dependency being present.
- CV screening latency is directly the candidate's page-load latency for the application form. If Azure OpenAI is slow, the application form is slow.
- If a real queue is ever introduced (there's a clear future use case here — moving AI screening off the request path), it would need actual Celery/RQ/similar wiring added, not just relying on the currently-listed dependencies.

Testing

There is no `pytest` /CI-wired test suite for the backend. What exists:

- `backend/test_api.py`, `test_full_flow.py`, `test_submit.py` — standalone scripts that exercise a *running* server (job creation → application submission → screening → approval), useful as manual smoke tests or as a reference for the request/response shapes, but they don't run in CI.
- `backend/tests/test_realtime_advance_topic.py` — the one genuine unit test (no `pytest` framework, run via `python tests/test_realtime_advance_topic.py`), covering the interview topic/turn-count state machine (`advance_topic()` in `realtime_interviewer.py`). If you change interview-flow logic, run this and consider extending it.

No frontend test suite was found either (`npm run test` in the CI workflow is a no-op — `--if-present` and no `test` script exists in `package.json`).

Known operational debt, summarized

Item	Risk if ignored
No Alembic, hand-rolled column migrations	Schema changes beyond "add a nullable column" require manual work and are easy to get wrong across SQLite/Postgres
Celery/Redis declared but unused	AI screening runs in-request; a slow AI provider directly slows down the public application form
No backend CI/CD workflow found	Production backend deployment process is undocumented — confirm and record it
No automated test suite (backend or frontend)	Regressions in the interview state machine or screening logic won't be caught before production
Two Supabase/backend data stores for jobs & applications	See 02-data-model-and-storage.md — silent data loss during backend outages